

sbac

Manual de usuario

1 Introducción.

1.1 ¿Qué es *sbac*?

sbac es una herramienta diseñada para el control de versiones y la administración de la configuración. Su objetivo principal es ser capaz de almacenar distintas versiones de un mismo archivo o del mismo proyecto.

1.2 ¿Por qué utilizar control de versiones?

Al trabajar en un proyecto, los archivos que lo conforman suelen ser modificados todo el tiempo, provocando problemas como: pérdida de información, imposibilidad de volver a utilizar versiones funcionales del proyecto, dificultad para identificar el momento donde un error aparece, etc.

sbac soluciona estos y otros problemas almacenando versiones organizadas del proyecto y permitiendo navegar entre ellas.

2 Instalación y requisitos.

Para utilizar *sbac*, sólo se necesita utilizar el lenguaje *Python*, las bibliotecas incluidas por defecto en dicho lenguaje son suficientes para ejecutar *sbac* sin mayores complicaciones.

Para poder utilizar esta herramienta, basta con tener el archivo *sbac.py* en cualquier ubicación accesible desde la terminal.

3 Creación de un repositorio.

3.1 Cómo se crea un repositorio.

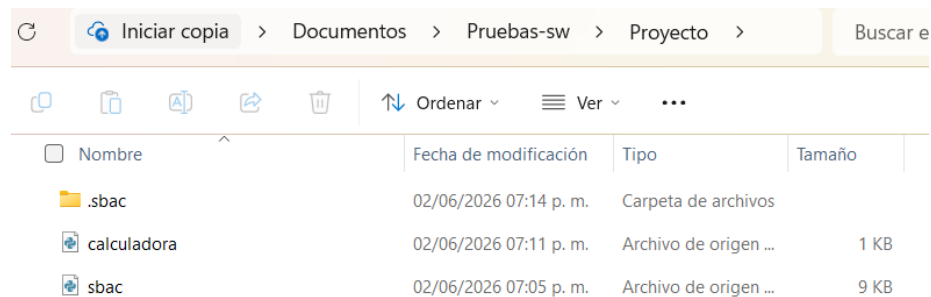
Una vez completado lo anterior, el siguiente paso es empezar a trabajar con *sbac* mediante la instrucción *python sbac.py start*.

3.2 Resultado esperado.

Al ejecutar el comando anterior, se debe obtener un resultado similar al siguiente:

```
PS C:\Users\ccela\Documents\Pruebas-sw\Proyecto> python sbac.py start
SBAC: ¡Repositorio inicializado con éxito en .sbac/!
```

donde la carpeta `.sbac/` relacionada al repositorio del proyecto se encontrará en el directorio donde se éste se ubica:



Nombre	Fecha de modificación	Tipo	Tamaño
.sbac	02/06/2026 07:14 p. m.	Carpeta de archivos	
calculadora	02/06/2026 07:11 p. m.	Archivo de origen ...	1 KB
sbac	02/06/2026 07:05 p. m.	Archivo de origen ...	9 KB

De este modo se habrá verificado la creación del repositorio.

4 Administración de archivos.

4.1 Adición al seguimiento de archivos.

Un archivo en seguimiento es aquel que se desea añadir en un futuro *commit* al repositorio del proyecto, es decir, se busca preservar el estado en el que se encuentre antes de realizar dicha acción. En el caso de *sbac*, el comando diseñado para registrar un archivo para seguimiento es `python sbac.py add "NombreDe-LArchivo"`.

Ejemplo:

Si tenemos el archivo *calculadora.py*:

```
Proyecto > calculadora.py > sumar
1  def sumar(a,b):
2      return a+b
3
4  def restar(a,b):
5      return a-b
```

Podremos añadirlo para seguimiento con el comando descrito anteriormente:

```
PS C:\Users\ccela\Documents\Pruebas-sw\Proyecto> python sbac.py add calculadora.py
SBAC: Se añadió 'calculadora.py' al seguimiento.
```

Del mismo modo, podremos añadir otros archivos para seguimiento de forma simultánea.

```
Proyecto > hola_mundo.py
1  print("Hola mundo")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\ccela\Documents\Pruebas-sw\Proyecto> python sbac.py add hola_mundo.py
SBAC: Se añadió 'hola_mundo.py' al seguimiento.
```

4.2 Consulta del seguimiento de archivos.

Si lo que se busca es averiguar cuáles archivos están en seguimiento, se deberá ejecutar el comando *python sbac.py status*.

```
PS C:\Users\ccela\Documents\Pruebas-sw\Proyecto> python sbac.py status
=== Estado del Repositorio ===
Última versión (HEAD): None
Archivos bajo seguimiento:
- calculadora.py
- hola_mundo.py
```

Este comando nos permitirá ver en forma de lista aquellos archivos a los que se les esté dando seguimiento y que posteriormente podrán ser añadidos al repositorio, además de mostrar al usuario cuál es la versión activa del proyecto.

5 Gestión de versiones (*commits*).

5.1 ¿Qué es un *commit*?

Un *commit* es un registro permanente en el repositorio de los archivos en seguimiento, conservando el estado de los mismos hasta el momento de realizar dicho registro.

Para realizar un *commit* en esta herramienta, se debe utilizar el comando *python sbac.py commit "Mensaje del commit"*.

```
PS C:\Users\ccela\Documents\Pruebas-sw\Proyecto> python sbac.py commit "Calculadora y Hola Mundo"
SBAC: Versión 'v1' guardada exitosamente.
PS C:\Users\ccela\Documents\Pruebas-sw\Proyecto> python sbac.py status
=== Estado del Repositorio ===
Última versión (HEAD): v1
Archivos bajo seguimiento:
- calculadora.py
- hola_mundo.py
```

5.2 Versiones y uso del mensaje como identificador.

Si bien *sbac* cuenta con un incremento automático de versiones con cada *commit*, puede ser algo confuso si se quiere encontrar una versión con una característica en particular, por lo que puede ser buena idea utilizar el mensaje que acompaña a cada *commit* como un identificador que nos señale qué caracteriza a ese *commit*.

Ejemplo: Si se modifica *calculadora.py*:

```
Proyecto > calculadora.py > restar
1  def sumar(a,b):
2      return a+b
3
4  def restar(a,b):
5      return a-b
6
7  def multiplicar(a,b):
8      return a*b
9
10 def dividir(a,b):
11     return a/b
```

El mensaje del siguiente *commit* puede ayudarnos a saber cuál fue el cambio realizado de forma sencilla.

```
PS C:\Users\ccela\Documents\Pruebas-sw\Proyecto> python sbac.py commit "Calculadora V2: Multiplicación y división añadidas"
SBAC: Versión 'v2' guardada exitosamente.
```

6 Consulta del historial.

Conforme nuestro proyecto recibe más y más modificaciones, podría interesarnos consultar todos los *commits* realizados sobre el repositorio del mismo. Con ese fin existe el comando *python sbac.py history*.

```
PS C:\Users\ccela\Documents\Pruebas-sw\Proyecto> python sbac.py history
=== Historial de Versiones ===
[v1] - 2026-06-02 19:28:11 | Mensaje: Calculadora y Hola Mundo
[v2] - 2026-06-02 19:30:21 | Mensaje: Calculadora V2: Multiplicación y división añadidas
```

El cual nos permitirá ver en forma de lista todos los registros con detalles como:

- Versión única asignada por el programa.
- Fecha.
- Hora.
- Mensaje elegido para representar a cada *commit*.

Datos útiles a la hora de entender por qué se realizó un cambio y en qué momento ocurrió.

7 Comparación y recuperación de versiones.

Una de las principales ventajas de utilizar un sistema de control de versiones es la posibilidad de analizar los cambios realizados a lo largo del tiempo y recuperar estados anteriores del proyecto cuando sea necesario.

7.1 Comparación de versiones.

A la hora de comparar versiones con el objetivo de observar los cambios entre ellas, *sbac* nos proporciona el comando `python sbac.py diff <version1> <version2>` donde los últimos dos argumentos serán las versiones asignadas por el programa.

Ejemplo:

Comparación entre las dos versiones de *calculadora.py*:

```
PS C:\Users\ccela\Documents\Pruebas-sw\Proyecto> python sbac.py diff v1 v2
=== Diferencias entre v1 y v2 ===

--- Cambios en: calculadora.py ---
--- calculadora.py (v1)
+++ calculadora.py (v2)
@@ -2,4 +2,10 @@
     return a+b

     def restar(a,b):
-        return a-b+    return a-b
+
+def multiplicar(a,b):
+    return a*b
+
+def dividir(a,b):
+    return a/b
--- hola_mundo.py: Sin cambios ---
```

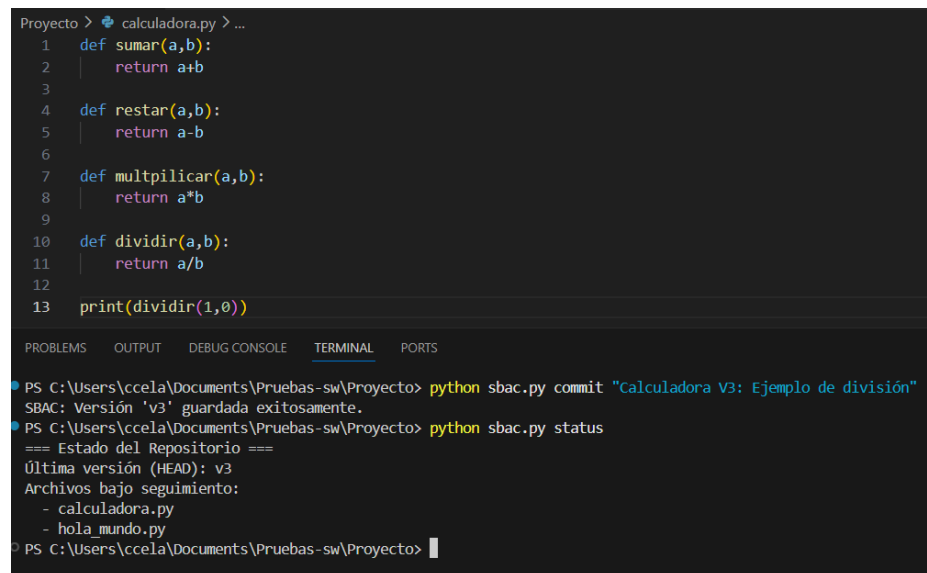
El resultado es la diferencia de contenido en ambas versiones señalada por símbolos distintos, mientras que *hola_mundo.py* se muestra sin cambios ya que no se efectuó ninguna modificación sobre dicho archivo.

7.2 Recuperación de versiones.

Por otra parte, si lo que se busca es volver a una versión antigua, pero estable, se puede utilizar el comando `python sbac.py checkout <version>` (donde nuevamente, el último argumento será la versión asignada previamente por el programa).

Ejemplo:

Si se añadiese algo a `calculadora.py` que pudiese afectar el desempeño del programa:



```
Proyecto > calculadora.py > ...
1  def sumar(a,b):
2      return a+b
3
4  def restar(a,b):
5      return a-b
6
7  def multiplicar(a,b):
8      return a*b
9
10 def dividir(a,b):
11     return a/b
12
13 print(dividir(1,0))

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
• PS C:\Users\ccela\Documents\Pruebas-sw\Proyecto> python sbac.py commit "Calculadora V3: Ejemplo de división"
SBAC: Versión 'v3' guardada exitosamente.
• PS C:\Users\ccela\Documents\Pruebas-sw\Proyecto> python sbac.py status
=== Estado del Repositorio ===
Última versión (HEAD): v3
Archivos bajo seguimiento:
- calculadora.py
- hola_mundo.py
PS C:\Users\ccela\Documents\Pruebas-sw\Proyecto> |
```

Se puede regresar a la última versión estable del archivo mediante la instrucción mencionada anteriormente:

```
Proyecto > calculadora.py > dividir
1 def sumar(a,b):
2     return a+b
3
4 def restar(a,b):
5     return a-b
6
7 def multiplicar(a,b):
8     return a*b
9
10 def dividir(a,b):
11     return a/b
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\ccela\Documents\Pruebas-sw\Proyecto> python sbac.py checkout v2
SBAC: Has regresado a la versión 'v2'. Los archivos en tu directorio han sido sobrescritos.

Permitiendo tomar medidas en contra de aquello que representaba un problema:

```
Proyecto > calculadora.py > dividir
5     return a-b
6
7 def multiplicar(a,b):
8     return a*b
9
10 def dividir(a,b):
11     if b == 0:
12         print("Error: La división entre 0 no está definida.")
13     else:
14         return a/b
15
16 print(dividir(4,2))
17
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\ccela\Documents\Pruebas-sw\Proyecto> python sbac.py commit "Calculadora V4 : Ejemplo de división válido"
SBAC: Versión 'v4' guardada exitosamente.
PS C:\Users\ccela\Documents\Pruebas-sw\Proyecto>

8 Líneas base (*baselines*).

8.1 ¿Qué es una *baseline*?

Una línea base o *baseline* es una etiqueta colocada sobre una versión existente que establece una referencia sobre una versión importante del proyecto.

8.2 ¿Cuándo utilizar *baselines*?

Las *baselines* se utilizan en contextos como: entrega al cliente, versiones estables, versiones de prueba, etc.

8.3 Etiquetado con *baselines* en *sbac*.

Con el comando `python sbac.py baseline <nombre>` podremos asignar una etiqueta a la versión **ACTUAL** del proyecto con el texto asignado al último argumento:

```
PS C:\Users\ccela\Documents\Pruebas-sw\Proyecto> python sbac.py baseline ESTABLE
SBAC: Línea base 'ESTABLE' asignada a la versión v4.
```

Si queremos revisar todas las líneas base relacionadas al proyecto, se deberá utilizar el comando `python sbac.py list-baselines`.

```
PS C:\Users\ccela\Documents\Pruebas-sw\Proyecto> python sbac.py list-baselines
=== Líneas Base Registradas ===
- ESTABLE -> [v4]
```

9 Lista de comandos.

- `python sbac.py start`: Inicializa un nuevo repositorio *sbac* en el directorio actual.
- `python sbac.py add <archivo>`: Agrega un archivo al seguimiento del repositorio.
- `python sbac.py status`: Muestra el estado actual del repositorio, incluyendo HEAD y archivos en seguimiento.
- `python sbac.py commit <mensaje>`: Crea una nueva versión (*commit*) de los archivos bajo seguimiento.
- `python sbac.py history`: Muestra el historial completo de versiones registradas.
- `python sbac.py baseline <nombre>`: Crea una línea base asociada a la versión actual.

- *python sbac.py list-baselines*: Lista todas las líneas base registradas en el repositorio.
- *python sbac.py diff <version1> <version2>*: Compara dos versiones y muestra las diferencias entre ellas.
- *python sbac.py checkout <version>*: Restaura una versión anterior y la convierte en la versión activa.